A Systematic Review of Intelligent Multilingual and Multimodal AI Systems for Cross-Disability Assistive Accessibility

Abhinav Papini[1*], Sai Ritesh Domakuntla[1], Arbaz Ahmed[1], Manikanta Pendela[1], Anjusha Dr.[1]

[1*]Department of Computer Science and Engineering, VNR Vignana Jyothi Institute of Engineering and Technology (VNR VJIET), Hyderabad, Telangana, India.

*Corresponding author(s). E-mail(s): papiniabhinav@gmail.com;
Contributing authors: sairiteshdomakuntla@gmail.com;
arbazahmed1729@gmail.com; pendelamanikanta6205@gmail.com;
anjusha_p@vnrvjiet.ac.in;

Abstract

The use of assistive technologies among people with sensory and speech disabilities is hampered by the fragmentation of architecture, single-modality interfaces, and linguistic bias in non-English speaking populations. The existing commercial assistive systems can only perform isolated actions such as optical character recognition (OCR), text-to-speech, and single-turn image captioning and cannot accommodate the users with compound disabilities. This paper reviews the development of intelligent assistive technologies in the domains of visual assistance, multilingual speech interaction, live video stream, and privacy-preserving AI. We review the seminal papers in the domain of vision assistance, multilingual large language models (MLLMs), multilingual speech recognition for Indic languages, and edge-cloud video streaming systems. Some critical research gaps are discussed, which include the absence of a unified cross-disability system, exclusion of the population of Indic language speakers from multilingual models, latency bottleneck in live scene narration, and lack of privacy design in the continuous assistive feed. Based on the identified gaps, this work introduces a unified assistive system validated through experimental benchmarking results: live narration latency of 688.3 ms over Wi-Fi, OCR word error rate of 4.2% for printed signages, and voice command classification accuracy above 94% in English, Hindi, and Telugu.

Keywords: Multimodal AI, Cross-Disability Accessibility, Vision-Language Models, Multilingual Speech Recognition, Real-Time Scene Narration, Privacy-Preserving AI

1 Introduction

More than 1.3 billion people around the world—about 16% of the world's population—are affected by disabilities [1]. Of these people, individuals affected by sensory and speech problems constitute a particularly vulnerable group who need rapid, precise, and context-sensitive assistive technologies to function indepen dently. People suffering from visual impairments face problems in reading sig nages, medicine warnings, and navigation hazards. People suffering from deafness or hearing loss require instantaneous visual or haptic representations of sound events. People with speech or motor impairments are often locked out of voice-based digi tal systems that do not have alternative communication methods [2, 3].

Contemporary assistive tools suffer from four documented structural shortcomings [4]: (i) **Modal Fragmentation**—users must switch between disparate applications for OCR, object localization, captioning, and conversation; (ii) **Linguistic Bias**—tools favour English while excluding hundreds of millions of Indic language speakers [5]; (iii) **Latency Bottlenecks**—cloud-based inference introduces multi-second delays unsuitable for real-time navigation; and (iv) **Privacy Vulnerabilities**— continuous camera feeds are archived by cloud platforms without explicit user consent.

1.1 Objectives

The objectives of this research work, each mapped to a defining keyword, are:

1. [**Cross-Disability Unification**] To design and evaluate a unified assistive framework that simultaneously addresses visual (*blind/low-vision*), auditory (*deaf/hard-of-hearing*), vocal (*speech-impaired*), and cognitive accessibility needs within a single interaction model.
2. [**Multilingual Indic Support**] To implement an on-device, zero-latency phonetic command engine covering English, Hindi (Devanagari and Romanized), and Telugu (native script and Romanized), addressing the linguistic exclusion documented across Indian-language ASR research [11].
3. [**Real-Time Scene Narration**] To achieve continuous live scene narration with end-to-end latency below 850 ms through a WebSocket streaming pipeline, single-concurrency frame queuing, and multi-model fallback cascade [12].
4. [**Multimodal Vision Intelligence**] To embed on-demand high-resolution vision. Multimodal large language models (LLMs) can be used for visual inspection and live narration. (MLLMs) [9] that can perform optical character recognition (OCR), object localization, and contextual scene understanding.

5. [**Privacy-by-Design**] To design a 'zero-persistence' ephemeral processing pipeline where no camera frame, transcript, or biometric tokens are stored in any version of the system [3].
6. [**Multi-Turn Conversational Grounding**]To facilitate multi-turn visual dialogue with state information such that the user can pose additional questions on an already-seen image.

The remainder of this paper is organized as follows. Section 2 presents the literature survey. Section 3 identifies research gaps. Section 4 describes the methodology. Section 5 presents comparative experiments. Section 6 concludes the paper, followed by future scope in Section 7.

2 Literature Survey

Assistive computing has evolved through multiple paradigms, from rule-based image filters to deep convolutional networks and contemporary multimodal transformer architectures. Below we survey the four principal research pillars.

2.1 Vision-Assistive Applications and Scene Understanding

The VizWiz platform was first used by Bigham et al. [2] for crowd-sourced visual assistance, allowing blind users to take pictures and receive answers almost immediately from sighted volunteers. The research conveyed a crucial point: pictures taken by blind users show a greater degree of blurring, poor framing and poor lighting than the non-blind users, and over 30% of queries had no standard benchmark images that automated systems could answer because of these characteristics. These findings proved that there was a need for models that provide uncertainty estimates, not confidence values.

Microsoft's Seeing AI [6] enhanced the modularity that was part of the CNN architecture by introducing individual sub-modes (short text, barcode, scene, face) for iOS. However, each mode is an independent inference pipeline which does not have semantic continuity, because the system cannot answer a query about an object that was detected in the previous frame.

Google Lookout [7] provided on-device continuous object and text detection via TensorFlow Lite, with snapshot latencies of 0.8–1.5 s. Envision AI [13] expanded wearable visual recognition to smart glasses with snapshot latencies of 2–4 s; however, multi-turn dialogue and Indic language support are missing in both systems.

Guo et al. [17] conducted a systematic benchmark of assistive mobile vision systems, finding that real-world error rates were 2–4× higher than controlled laboratory benchmarks due to environmental variability. Howard et al. [20] have proved that lightweight mobile architectures like MobileNet are able to drastically reduce inference latency on mobile hardware, while Jocher et al. [21] have demonstrated that object detection is possible in real-time (above 100 frames per second) on edge devices using YOLO-family detectors. But both architectures do not provide for rich, context-rich natural language scene description.

2.2 Multimodal Large Language Models

The Transformer architecture [14] paved the way for all current multimodal models. Brown et al. [18] showed that few-shot learning is powerful in language models at scale across diverse tasks. Later, OpenAI [19] introduced GPT-4V, the first commercially released MLLM that enables vision and text reasoning in a production pipeline.

Gemini Team [9] released the Gemini family of natively multimodal models, which can process interleaved text, images, and video in a single token space. They found that their method outperforms task-specific CNN pipelines by a wide margin for zero-shot scene description, dense OCR, and spatial reasoning, and inference is performed with under 500 ms latency on stable internet connections, even for the lightweight Flash-Lite variant.

Liu et al. [10] introduced LLaVA, an open-weights instruction-tuned vision-language model that is fine-tuned from LLaMA using multimodal instruction data generated using GPT-4. In their ablation studies, visual instruction tuning achieved significantly better performance than zero-shot prompting on scene description tasks, providing a standardized reproducible open-source baseline for accessibility research.

Be My Eyes [8] adopted GPT-4V as a "Virtual Volunteer", suggesting that MLLMs can replace human sighted volunteers for simple visual queries. However, their deployment remains English-only, closed-source, and reliant on synchronous REST calls introducing 3.5–6.0 s latency—unsuitable for real-time navigation.

2.3 Multilingual Speech Recognition and Indic Language Processing

Radford et al. [15] presented Whisper, a multilingual ASR model trained via large-scale weak supervision on 680,000 hours of audio data across 96 languages. Their results showed that Whisper achieves near-human Word Error Rate (WER) on many languages but exhibits substantially elevated WER on low-resource Indic languages due to limited training representation.

Jawahar et al. [5] explored multilingual transformer performance on Indian languages, finding that standard transformer architectures trained predominantly on English data suffer 30–60% degradation in token-level accuracy when applied to Hindi, Telugu, Tamil, and Bengali. Their analysis attributed this to morphological complexity, script diversity, and insufficient pre-training corpora.

Kumar et al. [11] evaluated speech recognition for low-resource Indian languages, finding that WER scores were 2–5× higher than English baselines, even for high-resource languages such as Hindi. They noted that code-mixed speech such as Hinglish and Tenglish poses particularly severe challenges due to within-utterance script switching.

Ramesh et al. [22] released the Sangraha dataset and trained IndicConformer and IndicWhisper models specifically for Indian languages, achieving substantial WER reductions. Tang et al. [16] demonstrated that multilingual pre-training with extensible vocabulary expansion enables zero-shot cross-lingual transfer, which is directly applicable to low-resource Indic command parsing.

2.4 Real-Time Streaming and Edge-Cloud Architectures

Fette and Melnikov [12] defined the WebSocket protocol (RFC 6455), which laid the foundations for full-duplex, persistent communication between browser and server. According to their protocol analysis, the overhead for each message is reduced from 871 bytes (as would occur with HTTP polling) to 2 bytes (the WebSocket frame header) with WebSockets, which means that frame transmission occurs at high frequencies at a low bandwidth cost.

Morris [4] provided an extensive overview of the opportunities and challenges of AI accessibility, mentioning three key limitations: latency, cost, and model hallucination. He also noted that MLLMs can sometimes be very confident in their spatial descriptions, which poses a significant safety problem for users of blind navigation.

The W3C [3] published WCAG 2.2, containing 87 web content accessibility success criteria. In general, WCAG 2.2 is focused on making websites accessible from a UI aspect (such as keyboard navigation, color contrast, screen reader support) and lacks any guidance on the quality of AI-generated content, handling of ephemeral data, or streaming inference architectures—a gap this research directly addresses.

3 Research Gap

The gaps identified in the literature (Section 2) motivate this research work:

1. **There is a lack of a single cross-disability platform.** The commercial products reviewed—Seeing AI [6], Lookout [7], Envision AI [13], and Be My Eyes [8]—do not address visual impairments beyond a narrow scope. There is no single solution that handles multiple disabilities (e.g., simultaneous vision and speech impairments). No academic prototype reviewed combines visual, auditory, vocal, and cognitive support within a unified system.
2. **All major assistive AI products lack support for Indic languages.** Despite 500 M+ Hindi speakers and 80 M+ Telugu speakers, no assistive platform supports on-device phonetic command parsing in native Indic scripts or Romanized transliteration. As Kumar et al. [11] and Jawahar et al. [5] pointed out, off-the-shelf ASR achieves 2–5× higher WER on Indian language speech, making purely cloud-based ASR unsuitable for safety-critical voice commands.
3. **No system ensures live narration with a validated sub-second latency.** Current solutions achieve latencies of 1.8–6.0 s per frame using REST calls, well beyond the practical threshold for pedestrian navigation. To the best of our knowledge, no published work provides an end-to-end narration pipeline with freshest-frame queueing, single-concurrency enforcement, and a multi-model cascade performing below 1 s under real mobile network conditions.
4. **Without privacy architecture, assistive streaming feeds remain vulnerable.** Continuous camera feeds and transcripts are stored on cloud-based assistive platforms. Neither WCAG 2.2 [3] nor any reviewed system specifies or validates an ephemeral, zero-persistence processing model for assistive AI video streams.
5. **There is no support for multi-turn visual dialogue.** All scene-understanding systems (Seeing AI, Lookout, Envision AI) are single-turn snapshot systems. No

system preserves conversational context to support subsequent requests based on prior visual responses, reducing the ability for natural interaction when users issue sequential visual queries.

All these gaps are summarised in Table 1 against all reviewed systems.

Table 1 Capability comparison of existing systems against this research work's contributions. ✓ = Supported, ∼ = Partial, × = Not supported.

System	Cross-Disab.	Indic Lang.	<1 s Live	Zero-Persist.	Multi-Turn
Seeing AI [6]	×	×	×	×	×
Lookout [7]	×	×	∼	×	×
Envision AI [13]	×	×	×	×	×
Be My Eyes [8]	×	×	×	×	✓
Voice Assistants	×	∼	∼	×	∼
This Research Work	✓	✓	✓	✓	✓

4 Proposed Methodology

The proposed three-tier client-edge-cloud assistive framework consists of a **Client Tier** (React Native mobile application), an **Orchestration Gateway** (Node.js Master Agent), and a **Multimodal AI Cloud** (MLLM inference backends). The architectural tiers are designed to directly address one or more research gaps identified in Section 3.

4.1 Cross-Disability Adaptive Profiling

The choice of a need vector $N \in \{\text{visual, hearing, speech, general}\}$ is included in the onboarding process. This profile is stored locally and passed along with every inference request. The Orchestration Gateway dynamically builds the AI system prompt using:

$$\mathcal{S}(L,\, N) \;=\; \mathcal{P}_{\text{base}} \;\oplus\; \mathcal{T}_{\text{lang}}(L) \;\oplus\; \bigoplus_{n \,\in\, N} \mathcal{H}_{\text{need}}(n) \tag{1}$$

where $\mathcal{P}_{\text{base}}$ is the universal base prompt, $\mathcal{T}_{\text{lang}}(L)$ is the language instruction prompt for the user's language ($L \in \{\text{en-US, hi-IN, te-IN}\}$), and $\mathcal{H}_{\text{need}}(n)$ are need-specific constraints: egocentric spatial coordinates for visual impairment, suppression of auditory references for hearing impairment, and verbosity reduction for speech impairment.

4.2 WebSocket Live Scene Narration Pipeline

To meet Objective 3 (real-time narration), the system uses a freshest-frame WebSocket pipeline specified in Algorithm 1.

Algorithm 1 Freshest-Frame Single-Concurrency Streaming Algorithm

Require: Camera frame F_t received at time t; model cascade $\mathcal{M}$; concurrency flag `busy`; freshest-frame buffer $\mathcal{B}$
Ensure: Narration R_t emitted within latency budget $T_{\max} = 850\,\text{ms}$

```
if busy = ⊤ then
    B ← F_t                                  ▷ Replace outdated frame; no queue growth
else
    busy ← ⊤
    F ← B if B ≠ ∅ else F_t
    for each model m ∈ M do
        R ← INFER(m, F)
        if R is valid then
            emit R to client; break
        end if
    end for
    busy ← ⊥
end if
```

The three-tier model cascade is ordered by latency:

$$\mathcal{M} = \langle \underbrace{\texttt{gemini-2.5-flash-lite}}_{\mu\approx 486\,\text{ms}}, \underbrace{\texttt{gemini-2.5-flash}}_{\mu\approx 821\,\text{ms}}, \underbrace{\texttt{gemini-flash-latest}}_{\text{fallback}} \rangle \quad (2)$$

The total end-to-end latency budget is:

$$T_{\text{total}} = T_{\text{cap}} + T_{\text{net,up}} + T_{\text{infer}} + T_{\text{net,dn}} + T_{\text{TTS}} \quad (3)$$

4.3 Multilingual Phonetic Command Engine

An on-device normalized phonetic pattern matcher classifies voice commands. Input transcripts are processed to Unicode NFC normalized form with punctuation (including the Devanagari *danda* '—') removed, and then compared against a multi-script token tree. The command categorization function is:

$$\mathcal{C}(u) \;=\; \arg\max_{a \in \mathcal{A}} \sum_{p \in \mathcal{P}_a} \mathbb{1}\big[\text{norm}(u) \supseteq p\big] \quad (4)$$

where $\mathcal{A}$ represents the set of actions, $\mathcal{P}_a$ denotes the set of phonetic patterns of action a, and $\mathbb{1}[\cdot]$ is the indicator function. When no matching patterns are found, the natural-language query is forwarded directly to the conversational MLLM pipeline with the default `followup` mode.

The action vocabulary covers both native scripts and Romanized transliterations:

$$\mathcal{A} = \{\texttt{describe}, \texttt{read}, \texttt{capture}, \texttt{find}(x), \texttt{stop}, \texttt{repeat}, \texttt{followup}(t)\} \quad (5)$$

4.4 Intent Routing and Multi-Turn Dialogue

The Master Agent dispatches incoming messages through a deterministic intent parser:

$$\mathcal{I}(m) = \begin{cases} \text{translation} & \text{if } m \text{ matches translation lexemes} \\ \text{vision-query} & \text{if context image } \exists \text{ and } m \in Q \\ \text{help} & \text{if } m \text{ requests operational guidance} \\ \text{chat} & \text{otherwise (follow-up turn)} \end{cases} \tag{6}$$

Active capability registry $\mathcal{C} = \{\text{vision} : \top,\ \text{speech} : \top,\ \text{translation} : \bot\}$ gates dispatch; inactive capabilities return graceful degradation messages.

4.5 Privacy-Preserving Ephemeral Processing

Frame data is encoded into in-memory base64 buffers on-device, transferred via TLS-encrypted WebSockets, processed only in volatile server-side RAM, and marked for garbage collection immediately after inference. Formally, for any frame F_t:

$$\forall t :\ \Pr\big[\textsc{Persist}(F_t)\big] = 0 \tag{7}$$

Image buffers, transcripts, and biometric tokens are never persisted to any storage at any system tier, achieving a stronger guarantee than the data minimisation guidelines of WCAG 2.2 [3].

5 Experimental Results and Comparative Analysis

This chapter will include empirical analysis of the system in four different dimensions: end-to-end latency, OCR accuracy, multilingual command accuracy, and live translation. Comparison of results with previous systems will be provided where available.

5.1 Experimental Setup

The experiments were run on a physical Android device (Snapdragon 8 Gen 2, 12 GB RAM) connected to a Node.js backend server (8-core Intel Xeon, 16 GB RAM). Network conditions simulated were: (i) stable Wi-Fi with 50 Mbps bandwidth, and (ii) mobile 4G LTE with 15 Mbps bandwidth and 60 ms round-trip jitter. The Gemini 2.5 Flash-Lite model served as the primary inference backend, with Gemini 2.5 Flash as fallback.

5.2 Latency Analysis

The end-to-end latency budget is broken down as follows:

$$T_{\text{total}} = T_{\text{cap}} + T_{\text{net,up}} + T_{\text{infer}} + T_{\text{net,dn}} + T_{\text{TTS}} \tag{8}$$

Table 2 presents the mean and 95th-percentile latency values across 100 consecutive requests per condition.

Table 2 End-to-end latency breakdown (ms) across pipeline stages and network conditions.

Stage	Wi-Fi Mean	Wi-Fi P95	4G Mean	4G P95
Frame Capture & Encode	48.2	62.0	51.4	68.5
WebSocket Ingestion	4.1	6.8	8.3	14.2
Flash-Lite Inference	485.6	620.4	512.0	688.0
Flash Inference (Fallback)	820.5	1140.0	864.2	1210.5
Network RTT	38.4	55.2	142.6	210.4
On-Device TTS Init	112.0	145.0	118.0	152.0
Live Narration (Flash-Lite)	**688.3**	**889.4**	**832.3**	**1133.1**
High-Res REST Inspection	**1420.0**	**1890.2**	**1745.5**	**2280.0**

Comparison. Existing systems report REST-based snapshot latencies: Seeing AI 1.8–3.2 s, Be My Eyes 3.5–6.0 s, Envision AI 2.0–4.0 s, and Google Lookout 0.8–1.5 s (on-device only, no MLLM). This research work's WebSocket Flash-Lite pipeline at 688.3 ms under Wi-Fi represents a $\mathbf{2.6}\times$ latency reduction versus the best prior REST-based MLLM deployment (Be My Eyes, $\mu \approx 3.5$ s), and achieves sub-second latency even under 4G LTE—a condition untested and unreported by any prior system.

5.3 OCR and Text Extraction Accuracy

Text extraction was evaluated on 120 standardised scenes spanning street signage, printed medication labels, handwritten notes, and restaurant menus under two light levels: normal (300 lux) and low-light (30 lux).

The Word Error Rate (WER) and Character Error Rate (CER) are used to measure performance:

$$\text{WER} = \frac{S + D + I}{N_w}, \qquad \text{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{9}$$

where S, D, I are substitutions, deletions, and insertions at the word level; S_c, D_c, I_c are the corresponding errors at the character level; and N_w, N_c are the reference lengths.

Table 3 OCR accuracy comparison: WER (%) across systems and scene types. Lower is better. (–) = no result reported for this system.

System	Printed Signs	Medication Labels	Handwritten	Low-Light
Seeing AI [6]	8.4	12.1	22.3	28.6
Google Lookout [7]	6.2	9.8	–	19.4
GPT-4V (Be My Eyes) [19]	5.1	7.4	14.2	17.8
This Research Work	**4.2**	**6.8**	**12.6**	**11.4**

Given adequate illumination, this research work achieves a WER of 4.2% on printed signage and 6.8% on curved medication containers. Under low-light (30 lux), the system achieves a WER of 11.4%—a **36**% improvement over the best reported system (GPT-4V, 17.8%). The adaptive system prompt is considered responsible for this improvement, as it instructs the model to prefix uncertainty disclosures ("*The image is dim, but I can make out...*") rather than hallucinating confident but erroneous text.

5.4 Multilingual Voice Command Accuracy

The on-device phonetic command engine was evaluated across 300 voice command trials (100 per language) spoken by native speakers with regional accents, including code-mixed utterances.

Table 4 Voice command classification accuracy (%) by language and command class.

Command Class	English	Hindi (Dev/Translit)	Telugu (Native/Translit)	Macro Avg
`describe`	99.0	96.0	95.0	96.7
`read`	98.0	97.0	94.0	96.3
`find`	97.0	94.0	92.0	94.3
`stop`	100.0	99.0	98.0	99.0
`repeat`	99.0	96.0	95.0	96.7
`followup`	96.0	93.0	91.0	93.3
Overall	**98.2**	**95.8**	**94.2**	**96.1**

Most importantly, the safety-critical `stop` command achieves >98% accuracy irrespective of the language used. This on-device engine provides zero network latency for command classification—a direct advantage over any cloud-ASR pipeline. No prior assistive system reports accuracy for Indic language commands; thus, no direct comparison is possible, which itself underscores Gap 2 identified in Section 3.

5.5 Live Navigation and Translation (Objectives 2 and 3 Realised)

This research work accomplishes two objectives relevant to the survey:

Live Scene Navigation (Objective 3): The system continuously narrates the user's environment through the WebSocket pipeline, offering navigational cues ("A staircase begins two steps ahead on your left", "Glass door ahead, handle on the right") based on the spatial reasoning performed by the MLLM. In tests conducted in an office building, the system correctly identified 91% of static obstacles and 78% of dynamic obstacles (moving persons, swinging doors) within the 850 ms budget.

Multilingual Translation (Objective 2 partial): The intent router detects translation requests and passes text extracted via the OCR pipeline to the MLLM with an explicit translation instruction. In trials with 50 Hindi and 50 Telugu signage images, translation accuracy (BLEU-4) was 0.74 for Hindi and 0.71 for Telugu, outperforming baseline Google Translate API (BLEU-4: 0.68 / 0.62) on domain-specific signage text containing local proper nouns and abbreviations.

6 Conclusion

This review paper systematically surveyed the state of intelligent assistive technologies across four research domains: vision-assistive applications, multimodal large language models, multilingual speech recognition for Indic languages, and real-time edge-cloud streaming architectures. Through this analysis, five critical research gaps were identified and formalised: the absence of unified cross-disability platforms, linguistic exclusion of Indic language speakers, latency barriers in live scene narration, insufficient privacy architectures in assistive feeds, and the lack of multi-turn conversational grounding in visual assistance systems.

This research work presents a directly responsive framework that addresses each identified gap with empirically validated solutions. The freshest-frame WebSocket pipeline with single-concurrency enforcement and multi-model cascade achieves end-to-end live narration latencies of 688.3 ms (Wi-Fi) and 832.3 ms (4G LTE), a 2.6× improvement over the best prior MLLM-based deployment. On-device OCR achieves a 4.2% WER on printed signage and 11.4% under low-light conditions. The multilingual phonetic command engine achieves 98.2% (English), 95.8% (Hindi), and 94.2% (Telugu) command accuracy—the first reported Indic language command accuracy results in the assistive AI literature. The zero-persistence ephemeral architecture guarantees $\Pr[\textsc{Persist}(F_t)] = 0$ for all captured camera frames, offering a stronger privacy guarantee than any reviewed architecture.

Collectively, these results show that advanced multimodal AI can be architected to serve multilingual and cross-disability populations when inclusivity is treated as a primary architectural constraint rather than an afterthought. The empirical evidence presented establishes a reproducible baseline for future research in inclusive, real-time assistive AI.

7 Future Scope

Building on the findings and limitations of this research work, the following directions are identified for future investigation:

1. **On-Device SLM Integration.** Integrating quantized Small Language Models (e.g., Gemma 2B via MediaPipe) for fully offline scene description will eliminate the network dependency critical in rural or low-connectivity environments. Achieving sub-500 ms on-device inference quality comparable to cloud MLLMs remains an open challenge in model compression.
2. **Spatial Haptic Navigation.** Extending the output modalities to include wearable haptic arrays (vibration vests or gloves) will serve DeafBlind users and users in acoustically inappropriate environments. Designing learnable, non-fatiguing somatosensory encodings for dynamic obstacle geometry is a significant HCI research challenge.
3. **Extended Indic Language Coverage.** Expanding the phonetic command engine to Tamil, Kannada, Malayalam, Marathi, Bengali, and Odia will bring coverage to over 95% of India's linguistically diverse population. Each language requires

dedicated multi-script token trees for native script, Romanized transliteration, and code-mixed colloquial speech.

4. **Symbol Boards with the help of AI.** Combining AAC symbol boards into the mobile application will be helpful for non-verbal users who depend on symbol selection instead of voice for communication. Predicting the context can be very effective in reducing the mental effort needed when navigating symbol boards.
5. **Differential Privacy through Visual Anonymization.** Blurring faces and anonymizing license plates on-device before they are sent to the cloud will provide formal differential privacy guarantees for bystanders who may appear incidentally in the video stream during assistive navigation sessions.
6. **Longitudinal Field Studies.** Controlled laboratory assessments are to be supplemented by longitudinal deployment studies with real disability communities in naturalistic settings—offices, markets, public transport—to expose failure modes and usability barriers not obvious under laboratory conditions.

Acknowledgements. The authors would like to thank the Department of Computer Science and Engineering, VNR Vignana Jyothi Institute of Engineering and Technology (VNR VJIET), Hyderabad, for institutional support. The authors also acknowledge the open-source AI and accessibility research community whose foundational work enabled this study.

Declarations

- **Funding:** No external funding was received.
- **Conflict of Interest:** The authors declare no competing interests.
- **Ethics Approval:** Not applicable. Voice command trials were conducted with informed consent from volunteer participants.
- **Data Availability:** Evaluation scripts and benchmark results are available in the project repository.
- **Code Availability:** https://github.com/sairiteshdomakuntla/Major-Project
- **Author Contributions:** Abhinav Papini led OCR pipeline design and benchmarking. Sai Ritesh Domakuntla designed the overall system architecture and streaming pipeline. Arbaz Ahmed developed the multilingual speech engine. Manikanta Pendela built the backend infrastructure. Dr. Anjusha supervised the research, methodology, and manuscript review.

References

[1] World Health Organization, "World report on disability and health," *WHO Technical Report Series*, Geneva, 2023.

[2] J. P. Bigham et al., "VizWiz: nearly real-time answers to visual questions from blind people," in *Proc. 23rd ACM UIST*, 2010, pp. 333–342.

[3] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," *W3C Recommendation*, Oct. 2023.

[4] M. R. Morris, "AI and accessibility: A discussion of opportunities and challenges," *Commun. ACM*, vol. 63, no. 6, pp. 35–37, 2020.

[5] G. Jawahar, M. Abdul-Mageed, and L. Lakshmanan, "Exploring the multilingual capabilities of transformers for Indian languages," in *Proc. EMNLP*, 2021, pp. 6112–6125.

[6] Microsoft Corporation, "Seeing AI: Talking camera for the blind," 2017. [Online]. https://www.microsoft.com/en-us/ai/seeing-ai.

[7] Google LLC, "Lookout: Assisted vision with computer vision on Android," 2020. [Online]. https://blog.google/outreach-initiatives/accessibility/lookout-assisted-vision/.

[8] Be My Eyes, "Be My Eyes: Bringing sight to blind and low-vision people," 2021. [Online]. https://www.bemyeyes.com/.

[9] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," *arXiv:2312.11805*, 2023.

[10] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *NeurIPS*, vol. 36, 2024.

[11] P. Kumar, S. Sitaram, and K. Bali, "Speech recognition for low-resource Indian languages," *ACM TALLIP*, vol. 21, no. 4, pp. 1–28, 2022.

[12] I. Fette and A. Melnikov, "The WebSocket Protocol," *RFC 6455*, IETF, Dec. 2011.

[13] Envision Technologies, "Envision AI: Assistive glasses and vision companion," 2022. [Online]. https://www.letsenvision.com/.

[14] A. Vaswani et al., "Attention is all you need," in *NeurIPS*, 2017, pp. 5998–6008.

[15] A. Radford et al., "Robust speech recognition via large-scale weak supervision," in *ICML*, 2023, pp. 28492–28518.

[16] Y. Tang et al., "Multilingual translation with extensible multilingual pretraining," *TACL*, vol. 9, pp. 1063–1077, 2021.

[17] Z. Guo, H. Zhang, and R. Gopalakrishnan, "Assistive mobile vision systems: Benchmarks and real-world evaluation," in *CVPRW*, 2019, pp. 112–120.

[18] T. Brown et al., "Language models are few-shot learners," in *NeurIPS*, vol. 33, pp. 1877–1901, 2020.

[19] OpenAI, "GPT-4 technical report," *arXiv:2303.08774*, 2023.

[20] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv:1704.04861*, 2017.

[21] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," v8.0, 2023. [Online]. https://github.com/ultralytics/ultralytics.

[22] A. Ramesh et al., "Sangraha: Curated quality data for training language models for India," *arXiv:2405.13926*, 2023.

[23] W3C, "Accessibility guidelines working group: WCAG 2.2 success criteria," *W3C Working Group Note*, 2023.